

Kubernetes-07

Taint and Tolerations:

=====

Taint and Tolerations are advance k8s pod scheduling technique.

Taint are attached to Node.

Toleration are attached to Pods.

Taint in general terms means a filter.

Tolerations with matching taint will be scheduled on tainted node.

Tolerations allow the scheduler to schedule pods with matching taints.

Example:

Nodes with special hardware. if you have nodes with special hardware (e.g GPUs) you want to repel Pods that do not need this hardware and attract Pods that do need it. This can be done by tainting the nodes that have the specialized hardware (e.g. `kubecttl taint nodes nodename special=true:NoSchedule`) and adding corresponding toleration to Pods that must use this special hardware.

How to apply taint to node ?

`kubecttl taint nodes <node_name> special=true:NoSchedule`

The taint has the format `<taintKey>=<taintValue>:<taintEffect>` .

Thus, the taint we just created has the key “special“, the value “true“, and the taint effect NoSchedule.

A taint’s key and value can be any arbitrary string and the taint effect should be one of the supported taint

effects such as NoSchedule , NoExecute , and PreferNoSchedule .

NoSchedule: No pod will be scheduled on the node.

PreferNoschedule: No guarantee that pod will not be scheduled on the node,there might be chance that pod might get scheduled.

NoExecute: If any pods are already running on tainted nodes then it will delete the running pods After adding taint, try to create pods then it will be created on other node only.

Taints and Tolerations (Control from Node side)

Purpose: To prevent some Pods from running on certain Nodes.

What is a Taint?

- Taint is applied on a **Node**
- It tells:
“Do NOT schedule normal Pods here”
- Only special Pods can run if they tolerate it

Effects of Taint

1. **NoSchedule** → Pod will NOT be scheduled
2. **PreferNoSchedule** → Try to avoid scheduling
3. **NoExecute** → Remove already running Pods

What is Toleration?

- Added in **Pod YAML**
- Allows Pod to run on tainted Node

Node Selector:

=====

Node selector helps us to select node where the pod can be scheduled.

It will work based on the label attached to node.

How to attach label to node ?

kubectl label nodes <node_name> env=prod

Node Selector (Simple method)

Purpose: Run Pod on a specific Node with a label.

Node Affinity:

=====

Node affinity is a set of rules used by the scheduler to determine where a pod can be placed.

Two types of Node Affinity is available:

1) PreferredDuringSchedulingIgnoredDuringExecution (Soft Scheduling):

Means we are asking scheduler to check if the values/label is matching then schedule pod on the node and

if node with same label is not available then it will schedule on any available node.

Ignored During Execution means if the pod is already scheduled on node and later on we have removed the label then

the pod will be ignored and continue running on the same node.

Node Affinity (Advanced version of Node Selector)

Same purpose as Node Selector
But more powerful and flexible

Two Types

1 —> **RequiredDuringSchedulingIgnoredDuringExecution**

- MUST match
- If no match → Pod not scheduled

2 —> **PreferredDuringSchedulingIgnoredDuringExecution**

- Try to match
- But not compulsory

When to Use What?

Use **Node Selector** → Simple cases

Use **Node Affinity** → Advanced matching

Use **Taints** → Reserve nodes (like GPU, DB, master)

Tasks:

Task 1: Execute all yaml files shown in video.

Task 2: Taint a Node and Schedule a Tolerant Pod Taint a node with `special=true:NoSchedule`. Create a pod with a toleration that matches the taint, allowing it to be scheduled on the tainted node.

Task 3: Use NodeSelector to Schedule a Pod on a Specific Node Label a node with `env=dev`. Create a pod with a nodeSelector that schedules it only on the node labeled `env=dev`.

Task 4: Use Node Affinity with Soft Scheduling Label a node with `env=test`. Create a pod with `PreferredDuringSchedulingIgnoredDuringExecution` node affinity, preferring to schedule it on a node labeled `env=test`. Remove the label and verify the pod continues to run.

Task 5: Implement Node Affinity with Hard Scheduling Create a pod with `RequiredDuringSchedulingIgnoredDuringExecution` node affinity, ensuring it will only be scheduled on a node labeled `env=prod`. Verify the pod cannot be scheduled if no node has the `env=prod` label.

Task 6: Taint a Node and Use NoExecute with Toleration Seconds Taint a node with `special=true:NoExecute`. Create a pod with a `tolerationSeconds` field (e.g., 60 seconds) and observe it gets evicted after 60 seconds on the tainted node.

Task 1:

Execute all yaml files shown in video.

PART 1 – Node Selector

Step 1: Check nodes

```
kubectl get nodes
```

```
[ubuntu@master:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE      VERSION
master      Ready     control-plane  7d21h    v1.30.14
worker1     Ready     <none>      7d20h    v1.30.14
worker2     Ready     <none>      7d20h    v1.30.14
ubuntu@master:~$
```

Step 2: Add label to worker1

```
kubectl label node worker1 env=prod
Check:
```

```
kubectl get nodes --show-labels
You should see env=prod on worker1.
```

```
worker2     Ready     <none>      7d20h    v1.30.14
[ubuntu@master:~$ kubectl label node worker1 env=prod
node/worker1 labeled
ubuntu@master:~$ kubectl get nodes --show-labels
NAME        STATUS    ROLES    AGE      VERSION    LABELS
master      Ready     control-plane  7d21h    v1.30.14    beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/d
ers=
worker1     Ready     <none>      7d20h    v1.30.14    beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=prod,kubern
worker2     Ready     <none>      7d20h    v1.30.14    beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/d
```

Step 2: Create YAML file

```
vi first.yml
```

```
ubuntu@master:~$ vi first.yml
[ubuntu@master:~$ cat first
cat: first: No such file or directory
[ubuntu@master:~$ cat first.yml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: firstcontainer
    image: nginx
    imagePullPolicy: Never
  nodeSelector:
    env: prod

ubuntu@master:~$
```

Step 3: Apply

```
kubectl apply -f first.yml
```

```
ubuntu@master:~$ kubectl apply -f first.yml
pod/firstpod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
firstpod  1/1     Running   0           24s   192.168.235.191  worker1   <none>           <none>
ubuntu@master:~$
```

Step 4: Verify

```
kubectl get pods -o wide
firstpod should run on worker1
```

PART 2 – Node Affinity (Soft Rule)

Step 1: Add label to worker2

kubectl label node worker2 env=test
Verify:

kubectl get nodes --show-labels

```
ubuntu@master:~$ kubectl label node worker2 env=test
node/worker2 labeled
ubuntu@master:~$ kubectl get nodes --show-labels
NAME        STATUS    ROLES    AGE   VERSION   LABELS
master      Ready     control-plane  7d21h  v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/ers=
worker1     Ready     <none>      7d21h  v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=prod,kuber
worker2     Ready     <none>      7d21h  v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=test,kuber
ubuntu@master:~$
```

Step 2: Create file

vi soft.yml

```
ubuntu@master:~$ cat soft.yml
apiVersion: v1
kind: Pod
metadata:
  name: softpod
spec:
  containers:
  - name: firstcontainer
    image: nginx
    imagePullPolicy: Never
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: env
            operator: In
            values:
            - test
ubuntu@master:~$
```

Step 3: Apply

```
kubectl apply -f soft.yml
```

Step 4: Verify

```
kubectl get pods -o wide  
softpod prefers worker2.
```

If worker2 has no label → it will run anywhere.

```
ubuntu@master:~$ vi soft.yml  
ubuntu@master:~$ kubectl apply -f soft.yml  
pod/softpod created  
ubuntu@master:~$ kubectl get pods -o wide  
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES  
firstpod   1/1     Running   0           5m51s  192.168.235.191  worker1   <none>            <none>  
softpod    1/1     Running   0           37s    192.168.189.125  worker2   <none>            <none>  
ubuntu@master:~$ cat soft.yml
```

PART 3 – Node Affinity (Hard Rule)

Step 1: Create file

```
vi hard.yml
```

```
[ubuntu@master:~]$ cat hard.yml
apiVersion: v1
kind: Pod
metadata:
  name: hardpod
spec:
  containers:
  - name: firstcontainer
    image: nginx
    imagePullPolicy: Never
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: env
            operator: In
            values:
            - test
ubuntu@master:~$
```

Step 2: Apply

```
kubectl apply -f hard.yml
```

Step 3: Check

```
kubectl get pods
```

If worker2 has `env=test` → hardpod runs

If not → hardpod stays **Pending**

```
ubuntu@master:~$ kubectl apply -f hard.yml
pod/hardpod created
ubuntu@master:~$ kubectl get pods
NAME         READY   STATUS    RESTARTS   AGE
firstpod     1/1     Running   0           14m
hardpod      1/1     Running   0           6s
softpod      1/1     Running   0           9m9s
ubuntu@master:~$ kubectl get pods -o wide
NAME         READY   STATUS    RESTARTS   AGE     IP             NODE       NOMINATED NODE
firstpod     1/1     Running   0           14m     192.168.235.191 worker1    <none>
hardpod      1/1     Running   0           20s     192.168.189.127 worker2    <none>
softpod      1/1     Running   0           9m23s   192.168.189.125 worker2    <none>
ubuntu@master:~$
```

Test IgnoredDuringExecution

After pod is running:

Remove label:

```
kubectl label node worker2 env-  
Check:
```

```
kubectl get pods -o wide
```

Pod will still be Running.

That proves:

IgnoredDuringExecution works

```
ubuntu@master:~$ kubectl label node worker2 env-  
node/worker2 unlabeled  
ubuntu@master:~$ kubectl get pods -o wide  
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
firstpod   1/1     Running   0           16m   192.168.235.191 worker1    <none>           <none>  
hardpod    1/1     Running   0           2m38s 192.168.189.127 worker2    <none>           <none>  
softpod    1/1     Running   0           11m   192.168.189.125 worker2    <none>           <none>  
[ubuntu@master:~$ kubectl delete pod hardpod  
pod "hardpod" deleted  
[ubuntu@master:~$ kubectl apply -f hard.yml  
pod/hardpod created  
[ubuntu@master:~$ kubectl get pods -o wide  
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
firstpod   1/1     Running   0           17m   192.168.235.191 worker1    <none>           <none>  
hardpod    0/1     Pending   0           5s    <none>        <none>       <none>           <none>  
softpod    1/1     Running   0           12m   192.168.189.125 worker2    <none>           <none>  
ubuntu@master:~$
```

If worker2 has `env=test` → hardpod runs

If not → hardpod stays **Pending**

Part 4 Taints & Tolerations

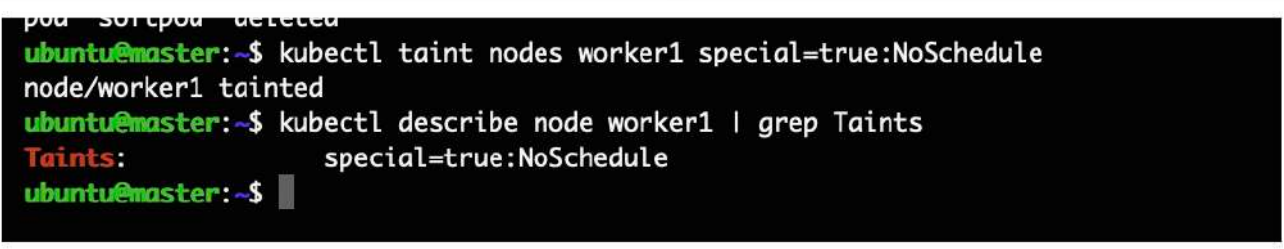
Your cluster:

- master
- worker1
- worker2

We will use **worker1** for taint.

STEP 1 – Apply Taint (NoSchedule)

```
kubectl taint nodes worker1 special=true:NoSchedule
```



```
pod --sortpod deleted
ubuntu@master:~$ kubectl taint nodes worker1 special=true:NoSchedule
node/worker1 tainted
ubuntu@master:~$ kubectl describe node worker1 | grep Taints
Taints:                special=true:NoSchedule
ubuntu@master:~$
```

Check:

```
kubectl describe node worker1 | grep Taints
```

You should see:

```
special=true:NoSchedule
```

Now worker1 is blocked

Meaning:

No new pod will be scheduled on worker1.

STEP 2 – Create Pod Without Toleration

Create file:

```
vi normal.yml
```

```
ubuntu@master:~$ vi normal.yml
[ubuntu@master:~$ cat normal.yml
apiVersion: v1
kind: Pod
metadata:
  name: normalpod
spec:
  containers:
  - name: nginx
    image: nginx

ubuntu@master:~$
```

Apply:

```
kubectl apply -f normal.yml
```

Check:

```
kubectl get pods -o wide
```

Pod will NOT go to worker1

It will go to worker2

Because worker1 is tainted.

```
ubuntu@master:~$ kubectl apply -f normal.yml
pod/normalpod created
ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          N
normalpod     1/1     Running   0           4s    192.168.189.65 worker2      <
ubuntu@master:~$ \
```

STEP 3 – Add Toleration (Operator: Equal)

Create file:

```
vi toleration-equal.yml
```

```
ubuntu@master:~$ vi toleration-equal.yml
[ubuntu@master:~$ cat toleration-equal.yml
apiVersion: v1
kind: Pod
metadata:
  name: tolerationpod
spec:
  containers:
  - name: nginx
    image: nginx
  tolerations:
  - key: "special"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"

ubuntu@master:~$
```

Apply:

```
kubectl apply -f toleration-equal.yml
```

Check:

```
kubectl get pods -o wide
```

Now this pod CAN run on worker1.

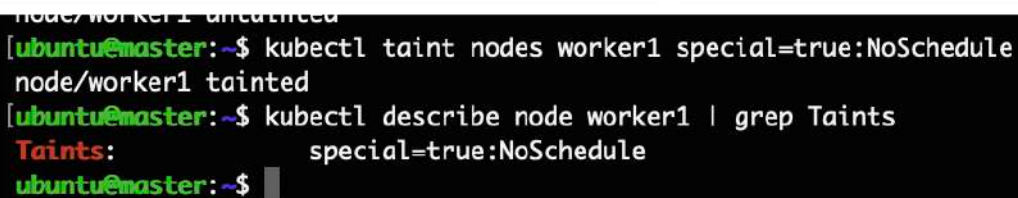
```
ubuntu@master:~$ kubectl apply -f toleration-equal.yml
pod/tolerationpod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE   NOMINATED
normalpod           1/1     Running   0           2m32s  192.168.189.65  worker2  <none>
tolerationpod       1/1     Running   0           4s    192.168.235.129  worker1  <none>
ubuntu@master:~$
```

TASK 2 : Taint a Node and Schedule a Tolerant Pod Taint a node with `special=true:NoSchedule`. Create a pod with a toleration that matches the taint, allowing it to be scheduled on the tainted node.

Taint a Node and Schedule a Tolerant Pod

Step 1: Taint worker1

```
kubectl taint nodes worker1 special=true:NoSchedule
```

A terminal window screenshot showing the process of tainting a node. The prompt is 'node/worker1 untainted'. The user runs 'kubectl taint nodes worker1 special=true:NoSchedule', and the output is 'node/worker1 tainted'. Then, the user runs 'kubectl describe node worker1 | grep Taints', and the output shows 'Taints: special=true:NoSchedule'. The prompt returns to 'ubuntu@master:~\$'.

```
node/worker1 untainted
[ubuntu@master:~]$ kubectl taint nodes worker1 special=true:NoSchedule
node/worker1 tainted
[ubuntu@master:~]$ kubectl describe node worker1 | grep Taints
Taints:                special=true:NoSchedule
ubuntu@master:~$
```

Step 2: Create Pod With Matching Toleration

```
vi first.yml
```

```

[ubuntu@master:~$ vi first.yml
[ubuntu@master:~$ cat first
cat: first: No such file or directory
[ubuntu@master:~$ .y
.y: command not found
[ubuntu@master:~$ cat first.yml
apiVersion: v1
kind: Pod
metadata:
  name: task2pod
spec:
  containers:
  - name: nginx
    image: nginx
  tolerations:
  - key: "special"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"

[ubuntu@master:~$

```

Step 3: Apply

```
kubectl apply -f first.yml
```

Step 4: Verify

```
kubectl get pods -o wide
```

Pod **can** run on worker1 (because toleration matches taint).

```

effect: NoSchedule

[ubuntu@master:~$ kubectl apply -f first.yml
pod/task2pod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE
task2pod      1/1     Running   0           5s    192.168.235.130  worker1    <none>
[ubuntu@master:~$

```

TASK 3 : Use NodeSelector to Schedule a Pod on a Specific Node Label a node with env=dev. Create a pod with a nodeSelector that schedules it only on the node labeled env=dev.

Use NodeSelector to Schedule a Pod on a Specific Node

Objective

1. Label a node with `env=dev`
2. Create a pod
3. Use `nodeSelector`
4. Pod must run **only** on that labeled node

Step 1 – Understand What We Are Doing

NodeSelector works like this:

—> Pod says:

“I want to run only on a node that has this label.”

If no node has that label → pod stays **Pending**.

STEP 1 – Add Label to worker2

We will use worker2.

```
kubectl label node worker2 env=dev
```

What This Does?

You are attaching metadata (label) to worker2.

Now worker2 has:

env=dev

This is just a key-value pair.

Verify Label

```
kubectl get nodes --show-labels
```

You should see env=dev under worker2



```
ubuntu@master:~$ kubectl label node worker2 env=dev
node/worker2 labeled
ubuntu@master:~$ kubectl get nodes --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
master	Ready	control-plane	7d21h	v1.30.14	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64
worker1	Ready	<none>	7d21h	v1.30.14	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64
worker2	Ready	<none>	7d21h	v1.30.14	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=dev,kubernetes.io/arch=amd64

```
ubuntu@master:~$
```

STEP 2 – Create Pod YAML

Now we tell pod:

"Only run where env=dev"

Create file:

```
vi task3.yml
```

```
[ubuntu@master:~$ vi task3.yml
[ubuntu@master:~$ cat task3.yml
apiVersion: v1
kind: Pod
metadata:
  name: task3pod
spec:
  containers:
  - name: nginx
    image: nginx
  nodeSelector:
    env: dev
ubuntu@master:~$
```

What This YAML Means

nodeSelector:

env: dev

This means:

- ✓ Find a node
- ✓ Check if it has label env=dev
- ✓ If yes → schedule
- ✓ If no → do not schedule

Simple filtering.

STEP 3 – Apply YAML

```
kubectl apply -f task3.yml
```

```
ubuntu@master:~$ kubectl apply -f task3.yml
pod/task3pod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE     IP             NODE
task2pod      1/1     Running   0           9m55s   192.168.235.130 worker1
task3pod      1/1     Running   0           7s      192.168.189.67  worker2
ubuntu@master:~$
```

STEP 4 – Verify Pod Location

`kubectl get pods -o wide`
Look at NODE column.

Pod should be running on **worker2**.

Extra Test (Very Important)

Now test understanding.

Remove label:

`kubectl label node worker2 env-`
Delete pod:

`kubectl delete pod task3pod`

Recreate:

`kubectl apply -f task3.yml`

Now check:

`kubectl get pods`

Pod will stay **Pending**

Why?

```
ubuntu@master:~$ kubectl label node worker2 env-  
node/worker2 unlabeled  
ubuntu@master:~$ kubectl delete pod task3pod  
pod "task3pod" deleted  
ubuntu@master:~$ kubectl apply -f task3.yml  
pod/task3pod created  
ubuntu@master:~$ kubectl get pods  
NAME      READY   STATUS    RESTARTS   AGE  
task2pod   1/1     Running   0           11m  
task3pod   0/1     Pending   0           6s  
[ubuntu@master:~$ kubectl get pods -o wide  
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES  
task2pod   1/1     Running   0           11m   192.168.235.130 worker1   <none>          <none>  
task3pod   0/1     Pending   0           15s   <none>       <none>    <none>          <none>  
ubuntu@master:~$
```

Because no node has `env=dev`.

That proves NodeSelector is strict.

TASK 3 Summary

You:

- ✓ Added label to node
- ✓ Created pod with nodeSelector
- ✓ Verified scheduling
- ✓ Tested failure case

TASK 4 : Use Node Affinity with Soft Scheduling
Label a node with `env=test`. Create a pod with `PreferredDuringSchedulingIgnoredDuringExecution` node affinity, preferring to schedule it on a node labeled `env=test`. Remove the label and verify the pod continues to run.

Use Node Affinity with Soft Scheduling

Objective

1. Label a node with `env=test`
2. Create a pod with **`PreferredDuringSchedulingIgnoredDuringExecution`**
3. Remove label
4. Verify pod still runs

First Understand Soft Scheduling

`PreferredDuringSchedulingIgnoredDuringExecution` means:

- Try to schedule on matching node
- If not available → schedule anywhere
- If label removed later → pod keeps running

It is a **soft rule**, not strict.

STEP 1 – Label worker2

```
kubectl label node worker2 env=test
```

Verify

```
kubectl get nodes --show-labels
```

Confirm worker2 has:

```
env=test
```



```
ubuntu@master:~$ kubectl label node worker2 env=test
node/worker2 labeled
ubuntu@master:~$ kubectl get nodes --show-labels
NAME        STATUS    ROLES    AGE     VERSION   LABELS
master      Ready     control-plane   7d22h   v1.30.14   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io
ers=
worker1     Ready     <none>        7d21h   v1.30.14   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io
worker2     Ready     <none>        7d21h   v1.30.14   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=test,kube
```

STEP 2 – Create YAML

```
vi task4.yml
```

```
[ubuntu@master:~]$ vi task4.yml
[ubuntu@master:~]$ cat task4.yml
apiVersion: v1
kind: Pod
metadata:
  name: task4pod
spec:
  containers:
  - name: nginx
    image: nginx
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: env
            operator: In
            values:
            - test

ubuntu@master:~$
```

What This Means

`preferredDuringSchedulingIgnoredDuringExecution`

- ✓ Scheduler prefers nodes with `env=test`
- ✓ But if not found → still schedules pod
- ✓ After scheduling → label removal does not affect pod

STEP 3 – Apply

```
kubectl apply -f task4.yml
```

STEP 4 – Verify

```
kubectl get pods -o wide
```

Pod should prefer **worker2**

```
pod task3pod deleted
ubuntu@master:~$ kubectl apply -f task4.yml
pod/task4pod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE   NOMINATEDNODE
task4pod      1/1     Running   0           10s   192.168.189.66 worker2 <none>
```

STEP 5 – Remove Label (Important Test)

```
kubectl label node worker2 env-
```

Now check:

```
kubectl get pods -o wide
```

Pod will STILL be running.

Because:

IgnoredDuringExecution
means Kubernetes will not disturb already running pod.

Why This Is Soft Rule?

If you delete pod and recreate it now:

```
kubectl delete pod task4pod
kubectl apply -f task4.yml
```

It will schedule on any available node.

Because `env=test` no longer exists.

```

ubuntu@master:~$ kubectl label node worker2 env-
node/worker2 unlabeled
ubuntu@master:~$ kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
task4pod  1/1     Running   0           94s   192.168.189.66 worker2    <none>           <none>
[ubuntu@master:~$ kubectl delete pod task4pod
pod "task4pod" deleted
[ubuntu@master:~$ kubectl apply -f task4.yml
pod/task4pod created
[ubuntu@master:~$ kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
task4pod  1/1     Running   0           4s    192.168.189.69 worker2    <none>           <none>
[ubuntu@master:~$ kubectl apply -f task4.yml

```

TASK 5 : Implement Node Affinity with Hard Scheduling

Create a pod with

RequiredDuringSchedulingIgnoredDuringExecution node affinity, ensuring it will only be scheduled on a node labeled **env=prod**. Verify the pod cannot be scheduled if no node has the **env=prod** label.

Implement Node Affinity with Hard Scheduling

Objective :

- Create a pod using **RequiredDuringSchedulingIgnoredDuringExecution**
- It must run **only** on a node labeled **env=prod**
- If no node has that label → Pod must stay **Pending**

This is a **hard rule**.

First Understand Hard Scheduling

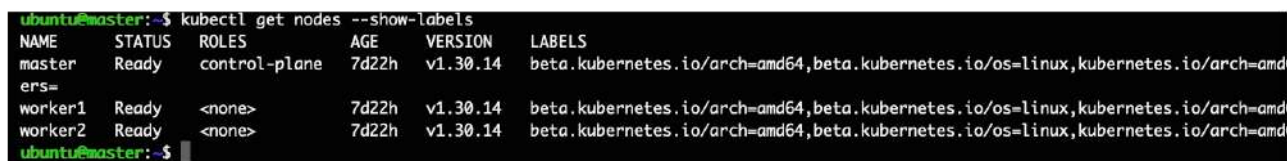
RequiredDuringSchedulingIgnoredDuringExecution means:

- ✓ Label MUST match
- ✓ If no match → Pod will NOT schedule
- ✓ After scheduling → removing label does NOT affect pod

STEP 1 – Make Sure No Node Has env=prod

Check:

```
kubectl get nodes --show-labels
```



```
ubuntu@master:~$ kubectl get nodes --show-labels
NAME        STATUS    ROLES    AGE   VERSION   LABELS
master      Ready     control-plane  7d22h v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=master,env=prod
worker1     Ready     <none>    7d22h v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker1,env=prod
worker2     Ready     <none>    7d22h v1.30.14  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker2,env=prod
ubuntu@master:~$
```

If any node has env=prod, remove it first:

```
kubectl label node worker1 env=
```

```
kubectl label node worker2 env=
```

Now no node should have env=prod

STEP 2 – Create YAML

```
vi task5.yml
```

```
[ubuntu@master:~$ vi task5.yml
[ubuntu@master:~$ cat task5.yml
apiVersion: v1
kind: Pod
metadata:
  name: task5pod
spec:
  containers:
  - name: nginx
    image: nginx
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: env
            operator: In
            values:
            - prod

ubuntu@master:~$
```

What This Means

`requiredDuringSchedulingIgnoredDuringExecution`

✓ Only schedule on node with `env=prod`

✗ If not found → Stay Pending

✓ After running → label removal won't affect

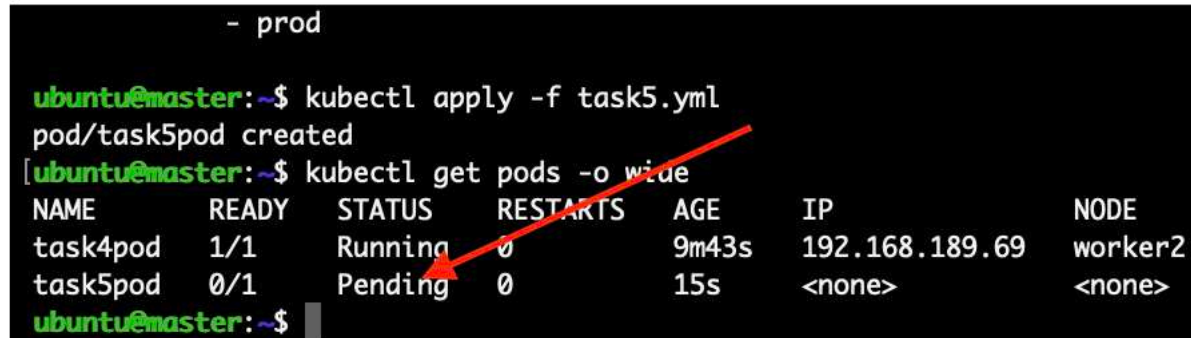
STEP 3 – Apply

```
kubectl apply -f task5.yml
```

Verify

```
kubectl get pods -o wide
```

```
- prod  
ubuntu@master:~$ kubectl apply -f task5.yml  
pod/task5pod created  
[ubuntu@master:~$ kubectl get pods -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE  
task4pod      1/1     Running   0           9m43s  192.168.189.69  worker2  
task5pod      0/1     Pending   0           15s    <none>          <none>  
ubuntu@master:~$
```



You should see:

```
task5pod    Pending
```

This is correct behavior.

Why?

Because no node has `env=prod`.

STEP 5 – Now Add Label and Test Again

Add label to worker1:

```
kubectl label node worker1 env=prod
```

Now check again:

```
kubectl get pods -o wide
```

```
ubuntu@master:~$ kubectl label node worker1 env=prod
node/worker1 labeled
ubuntu@master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
task4pod	1/1	Running	0	12m	192.168.189.69	worker2	<none>		<none>	
task5pod	0/1	Pending	0	3m19s	<none>	<none>	<none>		<none>	

Pod should now move to Running on worker1.

This proves hard scheduling works.

—> but still I was getting pending because from previous task we have not untainted the worker

FIX IT

Remove the taint:

```
kubectl taint nodes worker1 special=true:NoSchedule-
Now check:
```

```
kubectl get pods -o wide
```

task5pod should move to:

```
ubuntu@master:~$ kubectl taint nodes worker1 special=true:NoSchedule-
node/worker1 untainted
ubuntu@master:~$ kubectl get pods -o wide -w
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
task4pod	1/1	Running	0	15m	192.168.189.69	worker2	<none>		<none>	
task5pod	1/1	Running	0	6m	192.168.235.132	worker1	<none>		<none>	

Running on worker1

IMPORTANT LESSON

Scheduling checks happen like this:

- 1 Taints
- 2 NodeSelector / Affinity
- 3 Resources

Even if affinity matches,
taint can block scheduling.

Taints are stronger filter.

TASK 6: Taint a Node and Use NoExecute with Toleration Seconds Taint a node with `special=true:NoExecute`. Create a pod with a `tolerationSeconds` field (e.g., 60 seconds) and observe it gets evicted after 60 seconds on the tainted node.

Taint a Node with NoExecute + Use `tolerationSeconds`

Objective

1. Taint a node with `special=true:NoExecute`
2. Create a pod with toleration
3. Add `tolerationSeconds: 60`
4. Pod should run for 60 seconds
5. Then automatically get evicted

First Understand NoExecute

Effect: NoExecute

It does TWO things:

- ✓ Blocks new pods
- ✓ Removes already running pods

If pod has tolerationSeconds:

It stays for given seconds
Then Kubernetes removes it

STEP 1 – Taint worker1 with NoExecute

First make sure no old taints:

```
kubectl taint nodes worker1 special=true:NoSchedule-
```

Now apply NoExecute:

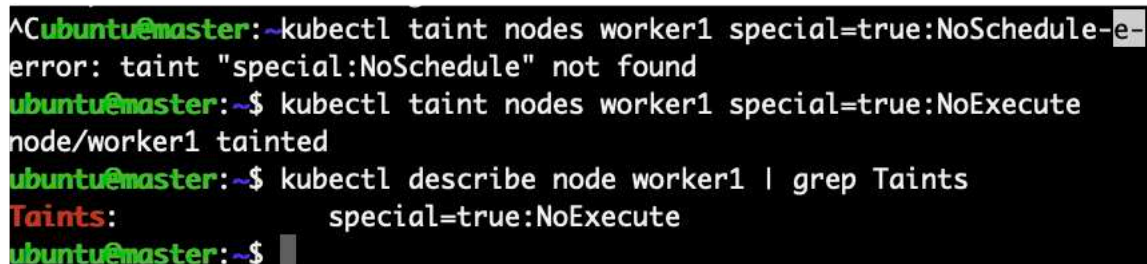
```
kubectl taint nodes worker1 special=true:NoExecute
```

Verify:

```
kubectl describe node worker1 | grep Taints
```

You should see:

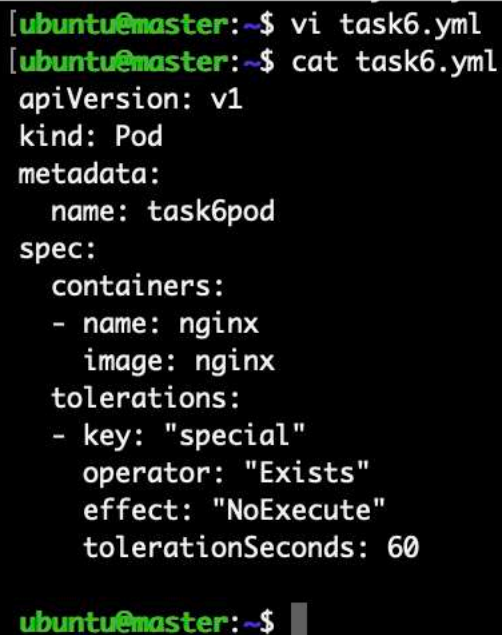
```
special=true:NoExecute
```



```
ubuntu@master:~$ kubectl taint nodes worker1 special=true:NoSchedule-e-  
error: taint "special:NoSchedule" not found  
ubuntu@master:~$ kubectl taint nodes worker1 special=true:NoExecute  
node/worker1 tainted  
ubuntu@master:~$ kubectl describe node worker1 | grep Taints  
Taints:                special=true:NoExecute  
ubuntu@master:~$
```

STEP 2 – Create Pod with tolerationSeconds

```
vi task6.yml
```

A terminal window with a black background and green text. It shows the commands 'vi task6.yml' and 'cat task6.yml' being executed. The output of 'cat' displays a YAML configuration for a Pod named 'task6pod' with a single container 'nginx' and a toleration for the 'special' node taint with an effect of 'NoExecute' and a duration of 60 seconds.

```
[ubuntu@master:~$ vi task6.yml
[ubuntu@master:~$ cat task6.yml
apiVersion: v1
kind: Pod
metadata:
  name: task6pod
spec:
  containers:
  - name: nginx
    image: nginx
  tolerations:
  - key: "special"
    operator: "Exists"
    effect: "NoExecute"
    tolerationSeconds: 60

ubuntu@master:~$
```

STEP 3 – Apply

```
kubectl apply -f task6.yml
```

STEP 4 – Watch Live

```
kubectl get pods -o wide -w
```

```
ubuntu@master:~$ kubectl apply -f task6.yml
pod/task6pod created
ubuntu@master:~$ kubectl get pods -o wide -w
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
task4pod	1/1	Running	0	27m	192.168.189.69	worker2	<none>		<none>	
task6pod	1/1	Running	0	4s	192.168.235.133	worker1	<none>		<none>	

Wait 60 sec

```
ubuntu@master:~$ kubectl apply -f task6.yml
pod/task6pod created
ubuntu@master:~$ kubectl get pods -o wide -w
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
task4pod	1/1	Running	0	27m	192.168.189.69	worker2	<none>		<none>	
task6pod	1/1	Running	0	4s	192.168.235.133	worker1	<none>		<none>	
task6pod	1/1	Running	0	60s	192.168.235.133	worker1	<none>		<none>	
task6pod	1/1	Terminating	0	60s	192.168.235.133	worker1	<none>		<none>	
task6pod	1/1	Terminating	0	60s	192.168.235.133	worker1	<none>		<none>	
task6pod	1/1	Terminating	0	60s	192.168.235.133	worker1	<none>		<none>	
task6pod	0/1	Terminating	0	60s	192.168.235.133	worker1	<none>		<none>	
task6pod	0/1	Terminating	0	61s	192.168.235.133	worker1	<none>		<none>	
task6pod	0/1	Terminating	0	61s	192.168.235.133	worker1	<none>		<none>	

You should see:

- 1 Pod becomes Running (likely on worker1)
- 2 After ~60 seconds → Status changes
- 3 Pod gets Terminated

What Is Happening Internally?

- Node has NoExecute taint
- Pod tolerates it for 60 seconds
- After 60 seconds → eviction controller removes pod

If tolerationSeconds not given:

-> Pod stays forever

Check status again:

kubectl get pod -o wide

```
[^Cubuntu@master:~$ kubectl get pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
task4pod   1/1     Running   0           34m   192.168.189.69 worker2    <none>           <none>
ubuntu@master:~$
```

Task6.pod is terminated

Step 5 : Remove the Taint

Once done, remove the taint to restore normal scheduling:

kubectl taint nodes worker1 special=true:NoExecute-

Behavior Summary

- Pods **without toleration** → immediately evicted
- Pods **with tolerationSeconds** → stay for given time, then evicted
- Pods **with permanent toleration (no tolerationSeconds)** → remain running indefinitely

```
[^Cubuntu@master:~$ kubectl get pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
task4pod   1/1     Running   0           34m   192.168.189.69 worker2    <none>           <none>
ubuntu@master:~$ kubectl taint nodes worker1 special=true:NoExecute-
node/worker1 untainted
ubuntu@master:~$
```

Conclusion – Task 6

Taint a Node with NoExecute and Use tolerationSeconds

In this task, we applied a taint with the effect **NoExecute** on a node and observed how pods behave with and without tolerations.

First, the node was tainted using:

```
special=true:NoExecute
```

This taint prevents new pods from being scheduled and also removes already running pods unless they have a matching toleration.

We then created a pod with a toleration that included:

- `effect: NoExecute`
- `tolerationSeconds: 60`

The pod was successfully scheduled on the tainted node and continued running for 60 seconds. After the specified time, Kubernetes automatically evicted (terminated) the pod.

This demonstrated that:

- Pods **without toleration** are immediately evicted.
- Pods **with tolerationSeconds** remain for the defined time and are then removed.
- Pods **with permanent toleration (without tolerationSeconds)** remain running indefinitely.

This task helped in understanding how Kubernetes controls pod placement and eviction using taints and tolerations, especially in scenarios such as node maintenance, failures, or special workload isolation

